

Nontrivial Relocation via a New *owning* reference Type

Document #: P2839R0
Date: 2023-05-12
Project: Programming Language C++
Audience: EWGI
Reply-to: Brian Bi
<bbi10@bloomberg.net>
Joshua Berne
<jberne4@bloomberg.net>

Contents

- 1 Abstract
- 2 Introduction
- 3 Structure of this proposal
- 4 Part I: Owning references and defaulted relocation constructors
 - 4.1 Summary
 - 4.2 Defaulted relocation constructors
 - 4.3 Conversions
 - 4.4 Relocation of automatic variables
 - 4.5 The `reloc` operator
 - 4.6 Reference collapsing and perfect forwarding
 - 4.7 Restrictions on owning references
 - 4.8 Further examples of disengagement and control flow
 - 4.9 Miscellaneous
 - 4.9.1 Implicit relocation by return statements
 - 4.9.2 Pseudo-destructor call on owning reference
 - 4.9.3 The `std::force_relocate` function
 - 4.9.4 The `std::relocate_ptr` smart pointer
 - 4.9.5 The `std::disengage` function

- 5 Part II: User-provided relocation constructors
 - 5.1 Example: Relocate-only small vector
- 6 Part III: Subordinate references and delayed initialization
 - 6.1 Motivation for delayed initialization
 - 6.2 Subordinate references
 - 6.3 Explicit owning reference to self for destructors
- 7 Comparison with other relocation proposals
 - 7.1 D2785
 - 7.2 N4158
 - 7.3 P0023
 - 7.4 P1144R7 and P2786R0
 - 7.5 P1029R3
- 8 Appendix A: Potentially breaking changes
- 9 Appendix B: Alternative syntaxes for forwarding references
 - 9.1 $\tau\sim$ syntax
 - 9.2 $\tau\&\&$ syntax
 - 9.2.1 Subapproach 1: Changing the reference collapsing rules
 - 9.2.2 Subapproach 2: Changing the reference collapsing rules but only for forwarding references
 - 9.2.3 Subapproach 3: Making reference collapsing ill formed outside forwarding references
 - 9.2.4 Subapproach 4: Abominable types
 - 9.3 A completely different syntax
 - 9.4 Conclusion on forwarding reference syntax
- 10 Appendix C: Alternative names
- 11 References

§ 1 Abstract

A new type of reference, known as the *owning reference*, is proposed with the spelling $\tau\sim$. An owning reference is responsible for destroying the object it refers to and may be used to initialize the parameter of a constructor of the form $\tau:\tau(\tau\sim)$, which is known as a *relocation constructor* and performs the responsibilities of both a constructor and destructor. An owning reference that has been moved from is *disengaged*, does not refer to an object, and is ill formed when named by an expression. Further extensions that build on

top of the basic concept of owning references are proposed to facilitate the implementation of user-defined relocation constructors.

§ 2 Introduction

Numerous proposals have attempted to introduce the ability to tie together the construction of one object with the destruction of a source object, migrating the value of that source object in the process. We discuss some such proposals in Section 7. Normal C++ move-initialization accomplishes migration of an object's value but fails to address the source object's lifetime, thus requiring that the source object support being in a *valueless* state (i.e., a state having a logically valid value that must be accounted for by any function having a wide contract, while semantically representing the absence of a value).

When an object's lifetime can also be ended as part of moving its value to a new object, performance can be improved in various ways:

- No writes need to be done to the source object to indicate that it is now valueless.
- For a type that requires a new allocation to be in a valueless state, such as some list implementations that allocate a sentinel node, this allocation can be completely skipped.
- No extra checks need to be done in a destructor to *clean up* an object that is in the valueless state.

On top of that, although not *all* move-initializations immediately precede the destruction of the source object, many of them do:

- Rearranging objects within a `std::vector` often involves chains of move-construct and destroy operations.
- Passing a function argument to a by-value parameter when the source object is a temporary generally involves nothing further happening with that temporary object prior to its destruction at the end of the complete enclosing statement.

Surprisingly, types that do not include references to themselves tend to not only be relocatable, but to be relocatable in a trivial fashion; i.e., the relocation can be accomplished by simply invoking `memcpy` on the source object and then not invoking that object's destructor but nonetheless ending its lifetime. This case is so prevalent and the performance benefits when taking advantage of it within containers is so significant that numerous historical proposals have been written to add *just* trivial relocation to the library or language: [P1029R1], [P1144R6], and [P2786R0].

A common initial response to these proposals for trivial relocation is confusion about proposing a trivial version of an operation where we do not have a nontrivial option. Certainly no other fundamental C++ operation is supported only when trivial and provides no mechanism to insert a user-defined version of that operation. This proposal aims to

provide the complete context in which to understand how trivial relocation could fit into a larger picture. In particular, should a trivial relocation proposal such as [P2786R0] move forward, it would be completely compatible with extending to arbitrary user-defined relocation through the owning references that we propose here.

By expressing relocation through owning references, rather than simply providing library functions that allow such relocation, we also extend the ability to leverage relocation in more places as well as to safely prevent one of the most common issues with move operations, use after move.

§ 3 Structure of this proposal

Our proposal is layered in three parts, with each part dependent on only the previous parts. Later parts could easily be delayed for future Standards while reaping a subset of the benefits and expressivity with a smaller initial feature.

Part I introduces owning references and the core language rules governing their behavior that are needed to support defaulted relocation constructors, which the authors believe will suffice for the vast majority of use cases that can benefit from nontrivial relocation.

Part II is a minimal extension to enable users to write their own relocation constructors. A new syntax is proposed to allow the compiler to track which subobjects have been relocated from within the *ctor-initializer* of a user-defined relocation constructor.

Part III builds on top of Part II to provide further usability benefits in the implementation of relocation constructors and to enable destructors to relocate subobjects.

Appendix A discusses extensions that depend only on Part I but are not included in any of the three main parts because of their potential impact upon existing code.

§ 4 Part I: Owning references and defaulted relocation constructors

§ 4.1 Summary

For every object type τ , we propose the introduction of a type called *owning reference to τ* , which is denoted by $\tau\sim$. An owning reference binds only to a value category known as the *rvalue*. An rvalue denotes an object that can be relocated from, just as an xvalue denotes an object that can be moved from. (See Appendix C for a discussion of alternative names.)

A value of owning reference type may be either “engaged” or “disengaged.” An engaged owning reference owns an object: When the owning reference’s lifetime ends, the object it owns is destroyed. (If the owning reference is a function parameter, the implicit destructor call at the end of the reference’s lifetime is performed in callee context because the caller

cannot know whether the callee has disengaged its owning reference parameter.) Whether a particular owning reference is engaged or disengaged at a particular program point is always known statically according to the rules that we will describe later in this section, and no runtime flags need to be maintained to track that status.

The name of a variable of type $\tau\sim$ is an lvalue, just like the name of any other reference variable. The lvalue can be converted to an rvalue using the `reloc` operator, which will be discussed in more detail later in this section. The resulting rvalue is then engaged, and the original variable is disengaged. An *id-expression* that names a disengaged owning reference is ill formed. If a variable of owning reference type is disengaged along some paths of control flow, it is implicitly disengaged at the end of all other branches (i.e., immediately before they rejoin the branch containing the explicit disengagement), as necessary, to ensure that it is known to be disengaged when the branches rejoin.

```
struct T {
    int m;
};

void g(T& x);

void f(T~ ref) { // `ref` is engaged and owns some object.
    g(ref); // OK; `ref` is an lvalue.

    T~ ref2 = reloc ref;
    // `ref` is disengaged; `ref2` is engaged and owns the object.

    g(ref); // ill formed; `ref` is disengaged
    ++ref.m; // ditto
    g(ref2); // OK

    if (rand() % 2) {
        {
            T~ ref3 = reloc ref2;
            // `ref3` is engaged and `ref2` is disengaged.
            // `ref3`'s lifetime ends here; `ref3.~T()` is called.
        }
        g(ref2); // error
    } else {
        g(ref2); // OK
        // `ref2` is implicitly disengaged here; `ref2.~T()` is called.
    }

    g(ref2); // error
}
```

See also Section 4.8 below.

A *relocation constructor* is a nontemplate constructor of a class τ whose first parameter is of type $\tau\sim$ and all of whose remaining parameters (if any) have default arguments. From the previous paragraph, it is apparent that when the constructor's parameter is initialized, the parameter has unique ownership of the object it refers to, and any other rvalue referring to the same object will have become disengaged. A relocation constructor, like

any other constructor, creates an object. Because the $\tau\sim$ parameter's lifetime ends at the closing brace of the constructor, the source object's lifetime will end by the time the relocation constructor returns.

§ 4.2 Defaulted relocation constructors

Certain types will have implicitly declared relocation constructors that are declared and defined in a similar manner to other special member functions but are unconditionally `noexcept`.

- For trivially relocatable class types, the declared constructor will be public. See Section 7.4 for discussion on what it means for a class type to be trivially relocatable.
- For class types `C` such that overload resolution for direct-initializing `C` from an xvalue of `C` succeeds and finds a nondeleted constructor and `C` has a nondeleted destructor, the declared relocation constructor will have the more restrictive of access levels of those two functions.

Throwing relocation constructors raise difficult specification problems. When a relocation constructor throws, some of the source object's subobjects will have been destroyed already, and destroying the remaining subobjects might not be safe because the order of destruction in a relocation constructor is opposite to the usual order of destruction (i.e., in a destructor). Like throwing move constructors, throwing relocation constructors are likely to cause problems for authors of generic code. For these reasons, so we do not propose to allow throwing relocation constructors at this time.

The behavior of an (implicitly or explicitly) defaulted relocation constructor is described in the following list.

- Trivially relocatable types have trivial relocation constructors that initialize the destination object as if by `std::memcpy`. As with any other relocation constructor, a trivial relocation constructor ends the lifetime of the source object. However, a trivial relocation constructor does not actually call the destructor for the source object, because the notion of trivial relocatability is that performing a bitwise copy followed by forgetting to destroy the old object has the desired semantics.
- Otherwise, if `C` is a class type described by the second bullet in the previous list:
 - If both the constructor selected for direct-initialization of `C` from an xvalue of `C` and the destructor of `C` are defaulted at their first declaration and if all direct subobjects and virtual base class subobjects of `C` are relocatable, the default relocation constructor for `C` performs a memberwise relocation of `C`'s direct subobjects and virtual base class subobjects. (The rationale is that we can assume that such a class does not need to be patched up after a memberwise relocation; any such required patchups would have to be performed by the constructor selected for move-construction, which would therefore have to be user provided.)

- Otherwise, the relocation constructor for `C` behaves as if it delegates to the constructor of `C` that would be selected to perform a move, which results in the subsequent implicit destruction of the source object when the `T~` parameter's lifetime ends:

```
T(T~ source) : T(static_cast<T&&>(source)) {}
```

- Otherwise, the relocation constructor is deleted. Note that the selection of a deleted function by overload resolution causes the program to be ill formed, but this outcome is undesirable when the function was implicitly declared or when it was explicitly defaulted in a templated class in which the author expects that it will sometimes be deleted. For move constructors and move assignment operators, C++11 solved this problem by specifying that if a defaulted move special member function turns out to be deleted, the function is ignored by overload resolution. We therefore propose that a defaulted relocation constructor that is defined as deleted is likewise excluded from the set of candidate functions in all contexts.

§ 4.3 Conversions

An rvalue of type `cv1 T` can be implicitly converted to an rvalue of type `cv2 T` if `cv2` is more cv-qualified than `cv1`.

An rvalue of type `T` can be implicitly converted to `T&&`. This conversion occurs automatically when the rvalue expression is the left operand of the `.` or `.*` operator.

A prvalue of object type `T` can be implicitly converted to an rvalue of type `T`, which has the effect of materializing a temporary that is owned by the resulting rvalue. During overload resolution, this conversion is considered better than binding to an rvalue reference or const lvalue reference. For example:

```
void foo(T~ r);    // 1
void foo(T&& r);   // 2

int main() {
    foo(T{});
}
```

A temporary of type `T` is materialized and converted to an rvalue. Then `// 1` is called, and `r` is bound to the resulting rvalue. When the owning reference `r` is destroyed at the end of its lifetime, it implicitly destroys the temporary object (unless `r` was disengaged prior to the end of its lifetime). The binding of the rvalue to the temporary object extends the storage lifetime for the object in the same manner as the binding of any other reference and suppresses the implicit destruction of the temporary object at the end of the full-expression in which it was created; the rvalue has ownership and is responsible for destroying the object.

There is no implicit conversion from `D~` to `B~`, where `B` is a base class of `D`. Allowing such a conversion would allow the referenced object to be passed to `B`'s relocation constructor, leaving the complete `D` object in a partially destroyed state. Since such a conversion is not

permitted, the implicit destructor call at the end of the lifetime of an engaged owning reference does not perform dynamic dispatch.

A glvalue of type T can be explicitly converted to $T\sim$ by `static_cast`. This generates an rvalue referring to the object that the glvalue refers to, which implies that this rvalue will be responsible for destroying the object, and the caller must ensure that they do not otherwise destroy the object. Such casts should therefore generally be used only with objects that have dynamic storage duration.

An rvalue of type T decays to simply T when deduced by value. An owning reference behaves like any other reference when named by a *simple-capture*. As when capturing a variable of object type, the programmer must ensure that the lambda closure object does not outlive the captured entity, lest the reference become dangling. A lambda closure object cannot have an owning reference member, for reasons that are discussed in Section 4.7.

```
struct T {
    int m;
};

template <class U>
void g(U u);

void f1(T~ ref) {
    g(reloc ref);
    // Calls `g<T>`, not `g<T~>`;
    // the parameter `u` is relocated from the object that `ref` refers
    // to.
    // `ref` is disengaged and the lifetime of the object `ref` refers
    // to ends.
    g(reloc ref); // ill formed
}

auto f2(T~ ref) {
    auto result = [ref] { return ref.m; };
    // The closure type has a member of type `T`, which is *copied* from
    // the
    // object that `ref` refers to.

    g(reloc ref); // OK; `ref` was not previously disengaged.

    return result;
}

auto f3(T~ ref) {
    auto result = [&ref] { return ref.m; };
    g(result); // OK

    g(reloc ref); // OK; `ref` was not previously disengaged.

    return result; // UB; the reference is now dangling.
}
```


§ 4.4 Relocation of automatic variables

We propose to change the current model of automatic variables to allow them to be relocated using the `reloc` operator. Automatic variables that are not passed to the `reloc` operator will continue to be implicitly destroyed upon scope exit, just as they always have been — though that mechanism now becomes defined in terms of owning references.

To accomplish these objectives, we propose that for each automatic variable, `x`, an implicit owning reference (call it `__x~`) is considered to be declared immediately after the locus of `x`'s declaration in the same scope. Immediately after its declaration, `__x~` is engaged and owns `x`. `x` is no longer inherently implicitly destroyed when it goes out of scope, but since `__x~` owns `x`, it will destroy `x` upon scope exit, unless some other owning reference takes over ownership of `x` first or an rvalue referring to `x` has been passed to a relocation constructor or otherwise disengaged. `__x~` cannot be named directly but is needed to define the `reloc` operator (see below). An *id-expression* naming `x` is ill formed if `__x~` is disengaged.

Although one might occasionally want to construct a new object in the storage location designated by `x` and re-engage `__x~` to that object, we do not currently propose to allow `x` to be named for such purposes, nor do we propose any method by which a disengaged owning reference can be re-engaged, because of the complexity of specifying such a feature and because the safety of doing so isn't clear. See Appendix A for more discussion.

```
struct T {
    int m;
};
int main() {
    T x = {0};
    T y;
    T~ r = reloc x; // `__x~` is disengaged; `r` owns `x`.
    ++x.m; // ill formed; `__x~` is disengaged.
    ++r.m; // OK; `r` is an lvalue.
    // `r` goes out of scope and destroys `x`.
    // `__y~` goes out of scope and destroys `y`.
    // `y` goes out of scope; `~T()` is not implicitly called.
    // `__x~` goes out of scope and does nothing since already disengaged.
    // `x` goes out of scope; `~T()` is not implicitly called.
}
```

§ 4.5 The `reloc` operator

The `reloc` operator is used to obtain an rvalue expression that owns a given entity and to disengage the previous owner. For these purposes, it may be applied to the following categories of *id-expressions*.

- When the operand is an *id-expression* (possibly parenthesized) denoting an automatic variable `x` of type `T~` belonging to a block scope or function parameter scope associated with the immediately enclosing function definition, the result is an rvalue referring to the object that `x` referred to; `x` is thereby disengaged.

- When the operand is an *id-expression* (possibly parenthesized) denoting an automatic variable x of object type belonging to a block scope associated with the immediately enclosing function definition, the result is `reloc __x~`.

We do not propose allowing `reloc` to be applied to a reference variable that is extending the lifetime of the temporary object it is bound to, because the reference might be to a subobject of the temporary object. See Section 4.3.

Because some ABIs require function parameters of object type to be destroyed on the caller side, applying `reloc` to the names of such parameters is not permitted in general; if the programmer wishes to relocate from a function parameter, they should ensure that the function parameter is declared with type $\tau\sim$ rather than τ . However, as an optional add-on to Part I, we propose adopting an idea from [D2785]: If τ is a relocate-only type (i.e., a type that has no eligible copy constructor and no eligible move constructor but does have an eligible relocation constructor), then it is permitted to relocate from a τ function parameter (implying that callee-destroy is required for such types, which currently do not exist).

§ 4.6 Reference collapsing and perfect forwarding

Because $\tau\sim$ is a reference type, there shall be no pointers to $\tau\sim$, references to $\tau\sim$, or arrays of $\tau\sim$. Writing out a type such as $\tau\sim\&$ directly is ill formed. However, owning references participate in reference collapsing.

- The composition of “owning reference to” and “lvalue reference to” (in either order) is “lvalue reference to.”
- The composition of “owning reference to” and “rvalue reference to” (in either order) is “rvalue reference to.”
- The composition of “owning reference to” and “owning reference to” is “owning reference to.”

These reference collapsing rules follow the “principle of lesser privilege” that currently governs the collapsing of lvalue and rvalue references. Owning references give the most privileges (the holder is permitted to destroy the object it refers to, possibly relocating its value to another object), followed by rvalue references (the holder is permitted to take ownership of the held resources, leaving the object in a moved-from state, but is not permitted to destroy the object), and lvalue references.

It follows that in the presence of owning references, forwarding references should be spelled “ $\tau\sim$ ”, where τ is a template parameter of a function that has a parameter of type $\tau\sim$. The template argument for τ is then deduced as an lvalue reference, rvalue reference, or nonreference when the function argument is, respectively, an lvalue, xvalue, or rvalue. (Since, as discussed in Section 4.3, a prvalue of type u prefers to be bound to $u\sim$ rather than $u\&\&$, using such a prvalue as the function argument will also result in τ being deduced as u .) A forwarding reference that is spelled $\tau\&\&$ can bind to an rvalue of type u but cannot forward it as an rvalue; the function parameter type will be $u\&\&$, not $u\sim$.

The issue of how to actually perform forwarding (which is typically done using an expression of the form `std::forward<T>(r)`, `static_cast<T&&>(r)`, or `static_cast<decltype(r)>(r)` in current C++) is thorny. When `r` is an owning reference, `reloc` must be used so that the disengagement of `r` that must be performed at the call site is visible to the compiler. However, it is essential to support a single syntax that perfectly forwards `r` regardless of whether it is an lvalue reference, rvalue reference, or owning reference; any alternative that would force users to implement a compile-time switch to call `reloc` on forwarding references of owning reference type — and an ordinary `static_cast` (or call to `std::forward`) in other cases — is not workable. For this reason, we propose to resurrect the proposal for a unary `>>` forwarding operator, which was described in [P0644R1] and rejected in Albuquerque (November 2017). When applied to a forwarding reference that is an owning reference, `>>` would be equivalent to `reloc`, and when applied to any other entity, `>>` would be equivalent to a `static_cast` as originally proposed. A function template that needs to perfectly forward one or more arguments would then take this form:

```
template <class T, class... Args>
foo<T> make_foo(Args... args) {
    return foo<T>(>> args...);
}
```

As an alternative to the `>>` forwarding operator, we propose to adopt an idea from [D2785], wherein `reloc` can also be applied to lvalue references and rvalue references, not only to the entities described in the previous section. Using `reloc` as the forwarding operator, the above function template could be written:

```
template <class T, class... Args>
foo<T> make_foo(Args... args) {
    return foo<T>(reloc args...);
}
```

The main disadvantage of `reloc` as the forwarding operator is that it would use the same keyword for two essentially *distinct* operators: an operator that disengages its operand to allow the compiler to track who owns a particular object and an operator that simply casts to lvalue reference or rvalue reference to facilitate perfect forwarding. To mitigate this disadvantage, we propose that when `reloc` is applied to an lvalue or rvalue reference, that operand shall be an owning reference. This restriction would not completely eliminate the inelegance and possible confusion arising from the use of `reloc` as the forwarding operator. For this reason, we believe that the unary `>>` operator would provide a better solution for perfect forwarding in the presence of owning references.

A third option is to specify that `static_cast<T~>(r)` implicitly applies `reloc` to `r` when `r` is a forwarding reference with declared type `T~`. The above function template could then be written:

```
template <class T, class... Args>
foo<T> make_foo(Args... args) {
    return foo<T>(static_cast<Args~>(args)...);
}
```

This syntax is much more verbose than the `>>` and `reloc` syntaxes, and would likely increase the popularity of `FWD` macros. We consider this outcome undesirable. We also believe that it is dangerous to allow the `static_cast` operator, which can accept any expression as an operand, to implicitly disengage its operand only when that operand has a very specific form. We do not propose this syntax for forwarding, but include it only for completeness.

We discuss some alternative specifications for forwarding references in Appendix B.

§ 4.7 Restrictions on owning references

A variable of owning reference type must have automatic storage duration. The purpose of this rule is to make it harder to accidentally create an owning reference that later becomes dangling. The rules we propose make it ill formed to reference an owning reference of automatic storage duration after it has become disengaged; there does not seem to be a similar strategy to prevent such unsafe accesses to owning references of static and dynamic storage duration. In the particular case of dynamic storage duration, there is a considerable risk that an owning reference attempts to destroy an object whose storage has already been released or reused (e.g., a variable of automatic storage duration whose block has already been exited).

Because owning reference variables are required to have automatic storage duration, they are not permitted as nonstatic data members. (The alternative — namely to make classes containing nonstatic data members of owning reference type ineligible to have any storage duration other than automatic storage duration — would create more problems than it solves.)

In addition, `~` is not permitted as a *ref-qualifier* in a function declarator; no variable could take ownership in such a case (considering that this is a pointer).

Explicit object parameters are permitted to have owning reference type. Note that calling a function with such an explicit object parameter will usually result in the implicit destruction of the object argument:

```
struct S {  
    /* ... */  
    void self_destruct(this S~ self);  
};  
S s;  
(reloc s).self_destruct();
```

We discuss an application for explicit object parameters of owning reference type in Part III.

Structured binding declarations are not permitted to have owning reference type; they suffer from the same issue as `~` on an implicit object member function: you can't actually name the entity to which the *ref-qualifier* applies (known as *e* in 9.6 [\[dcl.struct.bind\]](#)).

§ 4.8 Further examples of disengagement and control flow

If all flow-of-control paths through a particular branch result in a jump that exits the scope to which an owning reference belongs, the other branches do not implicitly disengage the owning reference. The reason for this exception to the usual implicit disengagement rules is that the branch containing the jump cannot rejoin the other branches, so implicit disengagement is not required in the other branches to prevent a situation in which the owning reference may or may not be disengaged after such rejoining.

```
struct T {
    void method();
};

void g(T);

T f(bool b) {
    T t;
    if (b) {
        return reloc t;
    } else {
        t.method(); // OK
    }
    return reloc t; // OK
    // `__t~` is disengaged and goes out of scope.
    // `t` goes out of scope.
}
```

A jump construct is not permitted to jump from a point where an owning reference is disengaged to a point that follows the definition of the owning reference but precedes an *id-expression* naming the owning reference. An implicit jump from the end of a loop back to its beginning is considered to occur.

```
int g(int~ r);

int f1(int~ r) {
    while (true) {
        g(reloc r); // ill formed
    }
}

int f2(int~ r) {
    while (true) {
        int x = 0;
        g(reloc x); // OK
        if (rand() % 8 == 0) {
            g(reloc r);
            return; // OK
        }
    }
}
```

When an rvalue expression is evaluated and is not otherwise disengaged by the end of the containing full-expression, the rvalue is implicitly disengaged as part of the last step in evaluating the full-expression; the timing of this implicit disengagement is the same as the timing of the implicit destructor call for a hypothetical temporary object that was created at the point at which the rvalue expression was evaluated. This implicit disengagement can occur, for example, when the rvalue expression is a discarded-value expression or when it is converted to an xvalue instead of being used to initialize an owning reference variable.

```
void f(T~ ref) {
    U{}, (reloc ref), V{};
    // `V` object destroyed, then `ref.~T()` called, and then `U` object
    // destroyed.
}
```

If an evaluation that disengages an owning reference variable is indeterminately sequenced or unsequenced relative to another evaluation in the same full-expression that names the owning reference variable (where an *id-expression* naming an automatic variable is considered to name its implicit owning reference for the purpose of this rule), the program is ill formed because we have no guarantee that the latter occurs before the former.

```
struct S {
    S(int x, int~ r);
};

void bar(int~ r) {
    S s1(r, reloc r); // Ill formed; `reloc r` may occur before the copy.
    S s2{r, reloc r}; // OK; `x` is copied from `r`, and then `reloc r`
    // is evaluated.
}
```

See Part II for an example in which this rule must be carefully understood.

Because the evaluation of a ternary conditional expression entails control flow, it performs implicit disengagement in the same manner as an if statement:

```
int bar(int~);

void foo(bool b) {
    int x = 0;
    int y = b ? bar(reloc x) : x;
    // OK; if `b` is false, `__x~` is implicitly disengaged after the
    // third
    // operand is evaluated.
    int z = x; // Ill formed; `__x~` is disengaged.
}
```

§ 4.9 Miscellaneous

Some of this section’s subsections propose new library facilities. The library facilities that will be proposed in a future revision of this paper should Part I move forward are not exhaustively enumerated herein.

§ 4.9.1 Implicit relocation by return statements

The `return x;` statements that currently implicitly move will behave as if by `return reloc x;` instead. Note that if no relocation constructor is available, the prvalue of $T\sim$ will implicitly convert to an rvalue of T , so the move constructor will be selected. The behavior of returning an object whose type does not have a relocation constructor (or whose type has a defaulted relocation constructor that is defined as deleted) will therefore be unchanged by this rule.

§ 4.9.2 Pseudo-destructor call on owning reference

If x is an *id-expression* (possibly parenthesized) naming an automatic variable of type $T\sim$ belonging to a block scope or function parameter scope associated with the immediately enclosing function definition, the expression `x.~T()` destroys the referenced object and disengages x . (This effect can also be achieved by evaluating `reloc x` in a discarded-value expression context, but the pseudo-destructor syntax is more evocative.)

§ 4.9.3 The `std::force_relocate` function

We propose that a library function, `std::force_relocate`, shall be provided by `<utility>`:

```
template <class T>
constexpr T~ force_relocate(T&& r) {
    return static_cast<T~>(r);
}
```

The `std::force_relocate` function can be used by, e.g., a `std::vector`-like container when reallocating. Let’s look at an example of how such reallocation can be performed. The reallocation does not suppress any implicit destructor call that would occur for its argument; the caller must remember not to destroy the source object separately.

```
template <class T>
void my_vector<T>::reallocate(size_type new_capacity) {
    T* new_buf = std::allocator_traits<Alloc>::allocate(alloc_,
        new_capacity);
    for (size_type i = 0; i < size_; i++) {
        ::new (static_cast<void*>(new_buf + i))
            T(std::force_relocate(buf_[i]));
    }
    capacity_ = new_capacity;
    buf_ = new_buf;
}
```

(Factory functions such as `std::allocator_traits<Alloc>::construct` should be updated to accept a pack of the new forwarding reference, `Args~...`. We have not yet

enumerated all Standard Library function templates to which this change should be made. After the Standard Library function templates are updated with this change, the above placement-new expression should be replaced by a call to `std::allocator_traits<Alloc>::construct`.)

§ 4.9.4 The `std::relocate_ptr` smart pointer

We propose a smart pointer type that is similar to `std::unique_ptr` but can be only relocated (not moved). Like `std::unique_ptr`, the smart pointer type guarantees that the deleter it holds will eventually be called to release the resources owned by the raw pointer it owns. However, while a `std::unique_ptr` can be accidentally dereferenced after it has been moved from (and become null), a `std::relocate_ptr` cannot be accessed in any way after it has been relocated from and omits the `release` and `reset` functions that can be used to change its value to null. We expect that `std::relocate_ptr` can be used in place of `std::unique_ptr` in most situations where `std::unique_ptr` is currently used, leading to safer code.

§ 4.9.5 The `std::disengage` function

We propose a library function, `std::disengage`:

```
template <class T>
constexpr void disengage(T~) requires is_object_v<T>;
```

Calling `disengage` (unsurprisingly) disengages the rvalue argument and ends the lifetime of the object to which it refers, *without* calling any destructors or relocation constructors. (Therefore, the effect of calling `disengage` is different from that of an *implicit* disengagement that occurs when `reloc` is applied to an owning reference or when the compiler inserts a disengagement along some branches of control flow; such implicit disengagements always call the destructor.)

```
#include <utility>

struct T {
    int m;
};

int main() {
    T x{1};
    T& r = x;
    std::disengage(x); // OK; `x.~T()` not called.
    int y = x.m;      // Ill formed; `x` is disengaged.
    int z = r.m;      // UB; dangling reference
}
```

Not particularly useful in Part I, the effect of `std::disengage` is purely to end the lifetime of the object to which the rvalue refers. This effect can be easily misused to subvert RAI but may be useful in user-provided relocation constructors; see Part II.

A user cannot implement `std::disengage` because it behaves as if it stashes away an owning reference in some place where the latter can live until the program terminates, which is not possible in user code since all owning references have automatic storage duration.

§ 5 Part II: User-provided relocation constructors

If Part I of this proposal is adopted, we expect that the vast majority of relocatable types will be trivially relocatable, and for the vast majority of nontrivially relocatable types, the defaulted relocation constructor (which will move then destroy) will do the right thing, because the necessary nontrivial work will already have been done when writing the move constructor and destructor. However, as relocate-only types become more common, so will class types that cannot be moved because they contain relocate-only subobjects. In some cases, patch-ups will need to be performed after memberwise relocation of these types, and since such types cannot be given a move constructor that performs the patch-ups, users must be able to write their own relocation constructor. In other words, if Part I is adopted without provisions to enable users to provide their own relocation constructors, relocation in C++ will become a victim of its own success. However, we propose Part II separately from Part I because specifying the semantics of user-provided relocation constructors involves additional complexities with less clear-cut solutions.

When users are allowed to write their own relocation constructors, the source object *must* not be implicitly destroyed, since the relocation operation takes the place of destruction. Therefore, the relocation constructor must ensure that each subobject of the source object is either relocated from or destroyed to avoid leaks. For usability and safety, we must ensure that destruction occurs automatically for each source subobject that is not relocated (i.e., the burden should not be on the user to remember to destroy them). (A relocation constructor thus offers the same guarantee with respect to its source object as a destructor, except it destroys the subobjects in the opposite order.) If the implicit destruction of subobjects that were not relocated does not occur in the *ctor-initializer*, then the body of the relocation constructor will see a source object that is partially alive. This situation is likely to result in unsafe code. The desire to prevent this situation leads to the conclusion that implicit destruction should occur in the *ctor-initializer*.

For the compiler to know which source subobjects to implicitly destroy, there must be a mechanism for the compiler to know which destination subobjects will be constructed by relocation from the corresponding source subobjects. The [D2785] approach in this area is for relocation constructors to implicitly relocate each destination subobject from the corresponding source subobject unless the subobject is explicitly named in the *ctor-initializer*. However, since we have owning references in our proposal, we can support more general constructors that do not have the exact signature `T::T(T~)`, and such constructors can have additional parameters as well. This raises the question of which such constructors should receive this implicit relocation treatment. We explain a use case for such extended relocation constructors below.

A possible extension (that we are not currently proposing) is to extend the `reloc` specifier (possibly spelled differently) to other kinds of parameters (typically for copy and move constructors), with the same meaning of “use this parameter as the default source for

initialization of subobjects” (i.e., by copy or move depending on the parameter type). The implicit disengagement would still apply to only owning references.

For the same reasons that we propose that relocation constructors be implicitly `noexcept` when implicitly declared or when explicitly defaulted on their first declaration (see Part I), we also propose that every constructor having a parameter that is declared `reloc` be implicitly `noexcept` and that it be a diagnosable error if such a constructor is declared `noexcept(false)`.

§ 5.1 Example: Relocate-only small vector

A *small vector* is a class template that provides inline storage for up to `N` objects of type `T`, where `N` is a template parameter and either employs dynamic memory allocation when the user attempts to store more than `N` objects or causes the operation to fail (e.g., by throwing an exception or terminating the program).

If Part I of this proposal is accepted, library authors might implement a `small_vector` template that supports relocate-only types. Such a `small_vector` would itself be relocate-only. Because a `small_vector` stores its elements inline, the relocation of a `small_vector` object invalidates all iterators into that object.

Consider now a struct `S` that holds both a `small_vector<T>` (where `T` is a relocate-only type) and an iterator into that `small_vector<T>`. `S` will not be trivially relocatable, since the iterator member must be patched up during relocation, nor will `S` have a usable defaulted relocation constructor, since it is not movable (see Section 4.2). The author of `S` must implement a relocation constructor:

```
struct S {
    small_vector<T>          d_v;
    small_vector<T>::iterator d_it;

    S(const S&) = delete;
    S& operator=(const S&) = delete;

    S(S~ src);
};
```

To relocate both `d_v` and `d_it` correctly to the destination object, the relocation constructor must compute the value `d_it - d_v.begin()` (call it `idx`) for the source object, and then initialize the destination’s `d_it` member with `idx + d_v.begin()`. Because `d_v` will be implicitly relocated by the *ctor-initializer*, the computation of `idx` cannot be deferred to the *compound-statement* of the relocation constructor. It follows that `S::S(S~)` needs to compute `idx` and then immediately delegate to another constructor that actually relocates `d_v`:

```
struct S {
    // other members previously described...

    S::S(size_t idx, reloc S~ src)
```

```

        : d_it(d_v.begin() + idx) {}

S::S(S~ src) : S{src.d_it - src.d_v.begin(), reloc src} {}
}

```

A number of features of the above implementation are noteworthy:

- When performing the delegation, we must use list-initialization syntax because parenthesized initialization would fail to provide the necessary sequencing guarantee to prevent `src` from being accessed after relocation (and would therefore be ill formed).
- Similarly, the code above will not compile if the two-parameter constructor has its parameters declared in the other order.
- If the `reloc` specifier is omitted for the two-parameter constructor, the compiler will attempt to default-initialize `d_v` and then destroy `src.d_v` at the end of the *ctor-initializer*, which is unlikely to be the semantics the programmer desired.
- Thanks to the `reloc` specifier, the construction of the `d_v` member by relocation from `src.d_v` is implicit. The explicit *mem-initializer* for `d_it` overrides the implicit relocation that would otherwise occur for `d_it`.

We believe that such subtleties make user-provided relocation constructors an expert-only feature, and even experts are likely to err. The additional features that we propose in Part III will simplify the implementation of `S` but at the cost of further complexity in the language specification.

§ 6 Part III: Subordinate references and delayed initialization

§ 6.1 Motivation for delayed initialization

The example given in Part II for a class containing a small vector of a relocate-only type can be rewritten much more simply if we introduce a feature that allows the construction of bases and members of the destination object to be deferred until some point in the *compound-statement* of its constructor. We propose that such deferred construction be performed by a new kind of statement called a *delayed-ctor-initializer*, consisting of this `:` and followed by a list of *mem-initializers* and terminated by a semicolon:

```

struct S {
    small_vector<T>          d_v;
    small_vector<T>::iterator d_it;

    S(const S&) = delete;
    S& operator=(const S&) = delete;

    S::S(reloc S~ src) {
        const size_t idx = src.d_it - src.d_v.begin();
        this : d_it(d_v.begin() + idx);
    }
}

```

```
}  
}
```

When the definition of a constructor contains a *delayed-ctor-initializer*, it shall not contain a *ctor-initializer* and shall not implicitly initialize bases and members prior to the constructor's *compound-statement*.

Since C++11, no compelling need for *delayed-ctor-initializers* in the language has arisen because delegating constructors can be employed as an alternative. We believe that the example from Part II, with its various gotchas, demonstrates a case in which delegation is particularly difficult to use correctly and difficult to read when used correctly due to the interaction of delegation with owning references. Thus, we propose *delayed-ctor-initializers* as part of this paper. Note that we propose to allow *delayed-ctor-initializers* in all constructors, not just relocation constructors. We believe that judicious use of *delayed-ctor-initializers* can result in less error-prone implementation of move constructors. (When programmers introduce a bug related to evaluation order in delegating move constructors, they will not receive a compile-time diagnostic as they would for a relocation constructor like the one discussed in Part II, so although *delayed-ctor-initializers* are important in supporting user-defined relocation constructors, *delayed-ctor-initializers* could end up being used more widely in move constructors than relocation constructors.)

To define a constructor in which control flow can pass through more than one *delayed-ctor-initializer* shall be a diagnosable error. To be more specific, suppose a hypothetical owning reference variable named `__r` were declared at the very beginning of the constructor's *compound-statement*, and each *delayed-ctor-initializer* were replaced by `reloc __r;`. The constructor is ill formed if the transformed version would be ill formed due to potentially referencing `__r` when `__r` is disengaged. Any implicit disengagement of `__r` and destruction of its referent that would occur due to the rules about owning references will instead result in the implicit execution of a *delayed-ctor-initializer* of the form `this : ;`.

§ 6.2 Subordinate references

The implicit relocation and disengagement semantics provided by the `reloc` specifier might not always be desired. In some cases, the programmer might wish for more explicit control. Consider, for example, an *allocator-extended* relocation constructor that does not use the allocator from the source object but instead uses an allocator supplied by the caller. That allocator must then be passed down to the allocator-extended relocation constructors of any subobjects that use allocators:

```
using allocator_type = ...;  
  
struct S {  
    allocator_type d_alloc;  
  
    S(S& src);  
    S(allocator_type alloc, S& src);  
};
```

```

struct T {
    allocator_type d_alloc;
    S              d_s;

    T(T~ src);
    T(allocator_type alloc, T~ src);
};

```

Implementing `T`'s allocator-extended relocation constructor using the tools provided by Part II is not possible because that constructor needs some way to execute a *mem-initializer* resembling `d_s(alloc, reloc src.d_s)`, but `src.d_s` isn't an *id-expression*, so under the rules in Parts I and II, `reloc src.d_s` is ill formed. As we explained in Part II, such constructs are not permitted because they make it impossible, in general, for the compiler to know which subobjects of the source object must be prevented from being destroyed a second time.

To allow this allocator-extended relocation constructor to be implemented, we need to specify the meaning of `reloc src.d_s`. The intuition behind the semantics of such an expression is that `reloc` acts upon an owning reference with a known declaration, so `src.d_s` must behave as if it *names* an owning reference variable, even though `src.d_s` is not one (nor could it be, since owning references are not permitted as members). Furthermore, if owning references are to exist to the subobjects of `src`, then at the point where such owning references exist, there must not be an owning reference to the complete object that is still planning on destroying it, lest the subobjects of `src` be destroyed twice.

We must therefore have an operation that acts upon the owning reference `src`, such that after this operation has been executed, `src` still refers to the same object as it did before, and referring to `src` is still well formed, but `src` no longer intends to destroy the object to which it refers. Such an owning reference is said to be *under destruction*. We chose this name because the state of an owning reference that is under destruction parallels the state of an object whose destructor has begun execution (namely, its base and member subobjects remain to be destroyed). The current “placeholder” syntax for this operation is `reloc_begin_destruction src`. (The identifier `reloc_begin_destruction` seems unlikely to have been used in real code but is unappealing; we hope to propose a better syntax eventually.)

The allocator-extended relocation constructor described at the beginning of this section can be implemented easily:

```

struct T {
    allocator_type d_alloc;
    S              d_s;

    T(T~ src);
    T(allocator_type alloc, T~ src) {
        reloc_begin_destruction src;
        this : d_alloc(alloc),

```

```

        d_s(alloc, reloc src.d_s);

        // `src.d_s` goes out of scope and was already disengaged.
        // `src.d_alloc` goes out of scope and is destroyed.
    }
};

```

The small vector example from the previous section can be rewritten so that it employs explicit relocation instead of the `reloc` specifier:

```

struct S {
    small_vector<T>          d_v;
    small_vector<T>::iterator d_it;

    S(const S&) = delete;
    S& operator=(const S&) = delete;

    S::S(S~ src) {
        const size_t idx = src.d_it - src.d_v.begin();
        reloc_begin_destruction src;
        this : d_v(reloc src.d_v),
              d_it(d_v.begin() + idx);

        // `src.d_it` goes out of scope and is destroyed.
        // `src.d_v` goes out of scope and was already disengaged.
    }
}

```

Control flow that potentially calls `reloc_begin_destruction` twice on the same owning reference is ill formed. Essentially, `reloc_begin_destruction r` is permitted only if replacing all such evaluations for a given `r` with `reloc r` would not result in any such invented `reloc r` violating the rules on use of owning references after disengagement. Implicit calls to `reloc_begin_destruction` are inserted as necessary (in a manner similar to implicit disengagements) to ensure that whether an owning reference is under destruction at a particular point is statically known.

`reloc_begin_destruction` is permitted outside of a constructor but only if every nonstatic data member, direct base class, and virtual base class of the operand would be accessible at the point where `reloc_begin_destruction` occurs.

`reloc_begin_destruction src` must not only place `src.d_s` under destruction, but also must initialize the *subordinate owning references* `src.d_alloc` and `src.d_s`. In general, there is one such subordinate owning reference for each direct nonstatic data member of object type and direct base class and one for each virtual base class if `src` is not itself a subordinate owning reference (i.e., it refers to a complete object). These subordinate owning references are declared in the same order in which the subobjects would be constructed so that when the subordinate owning references go out of scope, they destroy the corresponding subobjects in the same order in which the destructor of the complete object would destroy them.

At a particular point in the constructor, if `src` is under destruction, then a member access expression naming a member of `src`, whose left operand is the *id-expression* `src` (possibly parenthesized), is instead considered to name the subordinate owning reference corresponding to the named member. If `src` has a direct base class of type `B`, the syntax `static_cast<B&>(src)` names the subordinate owning reference corresponding to that base class subobject. (The syntax is not `static_cast<B~>(src)` because the result is an lvalue; this is consistent with the idea that the expression *names* an owning reference variable and that the name of an owning reference variable is always an lvalue referring to the owned object.)

```
struct S2 {
    std::string d_s1;
    std::string d_s2;
    std::string d_s3 = "not used for this object yet";

    S2(S2~ src) {
        reloc_begin_destruction src;
        // declares subordinate references to `src.d_s1`, `src.d_s2`,
        and
        // `src.d_s3`, in that order;
        // `src` is now under destruction

        this : d_s1(reloc src.d_s1)
            // disengages subordinate reference to `src.d_s1`
            , d_s2(reloc src.d_s2)
            // disengages subordinate reference to `src.d_s2`
            ;
            // initializes `d_s3` using its default member initializer

        std::cout << src.d_s1.size();
        // Ill formed: src.d_s1 names subordinate reference,
        // which is disengaged.
        std::cout << src.d_s3.size();
        // OK

        // Subordinate reference to `src.d_s3` goes out of scope and
        // destroys `src.d_s3`.

        // Subordinate reference to `src.d_s2` goes out of scope and is
        // already disengaged.

        // Subordinate reference to `src.d_s1` goes out of scope and is
        // already disengaged.

        // `src` goes out of scope, but it is under destruction so it does
        // not call `src.~S()`.
    }
};
```

The meaning of `src.d_s` depends on whether `src` is under destruction, so performing a member access through `src` at a point where control flow is ambiguous as to whether `reloc_begin_destruction src` has been evaluated is ill formed.

When a constructor containing a *delayed-ctor-initializer* also has a parameter that bears the *reloc* specifier, the implicit relocation and destruction semantics of the *reloc* specifier do not go into effect until the *delayed-ctor-initializer* is executed.

```
struct S1 {
    T d_foo;
    T d_bar;

    S1(S1~ source) {
        if (rand() % 2) {
            reloc_begin_destruction source;
            this : d_foo(source.d_foo),
                d_bar(0);
            // `source.d_foo` may no longer be referenced;
            // `source.d_bar` may still be referenced.
        }
        // implicitly:
        // else {
        //     reloc_begin_destruction source;
        //     this : ;
        // }
    }
};

struct S2 {
    T d_foo;
    T d_bar;

    S2(reloc S2~ source) {
        if (rand() % 2) {
            this : d_bar(0);
            // `d_foo` is implicitly initialized by relocation;
            // `source.d_foo` is destroyed by `T`'s relocation constructor
            // while `source.d_bar` is implicitly destroyed;
            // `std::disengage(reloc source)` is called implicitly.
        }
        // implicitly:
        // else {
        //     this : ;
        // }
    }
}
```

Evaluating *reloc_begin_destruction* for an owning reference parameter that is declared with the *reloc* specifier is ill formed. (The *reloc* specifier, described in Part II, implicitly performs a function that is very similar to *reloc_begin_destruction* prior to entering the *ctor-initializer*. However, note that the implicit destruction semantics afforded by the *reloc* specifier will destroy subobjects of the source object in the opposite order from the source object's destructor, while the subordinate references declared by *reloc_begin_destruction* will go out of scope in the same order as their corresponding subobjects would be destroyed by the source object's destructor.)

§ 6.3 Explicit owning reference to self for destructors

We propose that the syntax `~T(this T& self)` be permitted for declaring a destructor. Note that destructors are currently not permitted to use explicit object parameter syntax; e.g., `~T(this T& self)` is not permitted. We propose to permit a destructor to use explicit object parameter syntax solely in the case where the parameter is an owning reference to the class type to which the destructor belongs.

In a destructor so declared, `reloc_begin_destruction self` is implicitly executed at the beginning of the destructor's *compound-statement*, and an *id-expression* naming a direct member of the destructor's class implicitly names a subordinate owning reference. That is, if `m` is a direct member of the destructor's class, either `m` or `self.m` can be used to name the subordinate owning reference. The syntax `static_cast<Base&>(self)` must be used to name the subordinate owning reference corresponding to a direct base class subobject of type `Base`.

The motivation for such destructor declarations is to permit a destructor to return a resource to a pool by relocation, where that resource is owned by a member of the destructor's class:

```
struct S {
    relocate_ptr<Resource> d_resource;
    ResourcePool*          d_pool;
    std::string             d_name;

    ~S(this S& self) {
        // `reloc_begin_destruction self` is executed implicitly.
        d_pool->return(reloc d_resource);
        // `d_resource` is equivalent to `self.d_resource`.
        // `d_pool` takes ownership of subordinate owning reference.

        // Subordinate owning reference corresponding to `d_name` goes out
        // of scope
        // and destroys `d_name`.

        // Subordinate owning reference corresponding to `d_pool` goes out
        // of scope.

        // Subordinate owning reference corresponding to `d_resource` is
        // already disengaged.

        // `self` is under destruction, so it does not attempt to re-
        // destroy the
        // object to which it refers.
    }
};
```

Our reason for proposing to make this feature available only in the presence of an explicit object parameter is to avoid giving special meaning to the expression `*this`, which would need to be evaluated to name a subordinate owning reference to a base class subobject — i.e., as part of the expression `static_cast<Base&>(*this)`. Since `*this` is not an *id-expression*, for `*this` (but not a more complex expression) to be usable for naming subordinate owning references would be counterintuitive.

All destructors that have such an explicit object parameter of owning reference type are considered prospective (just like destructors with no parameters) until the end of the class definition. Overload resolution is then performed among all destructors that have an explicit object parameter to select the one that is the most constrained. If both a selected destructor with no parameters and a selected destructor with an explicit owning reference parameter are present, the class definition is ill formed. If the selected destructor has an explicit owning reference parameter, any (explicit or implicit) call to that destructor implicitly applies `reloc`, as necessary, to initialize the destructor's parameter.

§ 7 Comparison with other relocation proposals

WG21 members have created many proposals for relocation in C++. We will first discuss the other known proposals for *nontrivial* relocation and explain why we are proposing the introduction of owning references while the other nontrivial relocation proposals make do without them. Afterward, we will discuss the interaction of our proposal with the two most recent *trivial* relocation proposals, which are known to be actively pursued by their authors.

§ 7.1 D2785

[D2785] is most similar to our proposal and introduces no additional types but does introduce a new kind of prvalue obtained from relocating a glvalue: a prvalue that already has storage backing it (as opposed to one that will construct an object into the storage determined by the context). In effect, D2785 also proposes a new value category but does not propose a generalized vocabulary for manipulating expressions of this value category. The type `T~` in our proposal is a reification of the fourth value category, just as `T&&` is a reification of the xvalue category that was introduced in C++11.

We believe that the introduction of the rvalue category — and of owning reference types that may bind to them — results in a conceptually simpler model than the model of D2785.

Owning references also provide practical benefits. Because an owning reference can be perfectly forwarded with only the runtime cost of copying a pointer and the actual relocation only occurs at the end of this process, our approach never requires intermediate relocations when multiple function calls intervene between the scope in which a source object is declared and the scope in which a destination object is constructed by relocation from that source object:

```
void consume(T t);

void logAndConsume(T~ r) {
    std::cout << "Eating: " << &r << std::endl;
    consume(reloc r); // calls relocation constructor
}
```

```
void f() {
    T src;
    logAndConsume(reloc src);
}
```

Functions with $T\sim$ parameters in our approach are expected to be declared with parameters of type T in the D2785 approach:

```
void consume(T t);

void logAndConsume(T r) {
    std::cout << "Eating: " << &r << std::endl;
    consume(reloc r); // calls relocation constructor
}

void f() {
    T src;
    logAndConsume(reloc src); // may call relocation constructor
}
```

In the above snippet, the creation of a new T object named r when calling `logAndConsume` can be elided if that function is inlined or if that function is given an ABI in which the T parameter is implicitly passed by reference, which is not possible in general. (The implementation decision to use such an ABI would necessarily affect *all* functions with the same signature as `logAndConsume`.) Giving users the ability to explicitly declare parameters to have type $T\sim$ gives them a way to select which ABI they want and also avoids the issue in the D2785 approach wherein the value of `&r` depends on whether r has been elided by the implementation.

§ 7.2 N4158

The fundamental relocation operation in [N4158] is a call to a customization point called `uninitialized_destructive_move`, which takes two pointer arguments called `from` and `to` and constructs an object `at` to having the value held by `*from` while also ending the lifetime of `*from`.

A pure library facility such as that proposed by N4158 cannot be used to relocate automatic variables because the call to `uninitialized_destructive_move` does not suppress the implicit destructor call when the variable goes out of scope. Therefore, the N4158 approach is necessarily pointer-based, while our approach is value-based and enables a natural coding style where objects that are to be relocated can be declared as local variables of object type.

In N4158, a programmer can avoid heap allocation for objects that are to be relocated by constructing them into a stack buffer but must then ensure that if the relocation actually occurs, the object is not thereafter accessed, and that if the relocation does not occur, the object's destructor is eventually called to release any resources owned by the object. In our approach, by declaring the object as an automatic variable, the necessary guarantees are

provided by the compiler. Our approach is therefore safer than N4158 because, by making most such accesses ill formed, it prevents accidental access to objects that might have been relocated from. (However, we are not proposing a borrow checker for C++; an lvalue reference to a local variable that is then relocated from can still be used to attempt to perform an access of that variable, resulting in undefined behavior.)

The N4158 approach encourages the programmer to pass the source object by pointer until the point at which the relocation will actually occur; doing so avoids unnecessary intermediate relocations, but the *intent to relocate* cannot be perfectly forwarded when the source pointer is passed, since it will appear to a factory function as just a pointer, and the factory function will pass that pointer to a constructor rather than calling `uninitialized_destructive_move`. In our approach, rvalues can be perfectly forwarded by functions that have a forwarding reference parameter spelled `T~`, and no additional machinery is required.

The `uninitialized_destructive_move` function proposed by N4158 could be implemented as follows under our proposal:

```
template <class T>
void uninitialized_destructive_move(T* from, T* to) {
    ::new (static_cast<void*>(to)) T(static_cast<T~>(*from));
}
```

Note that customization of the functionality of the implementation shown above would be accomplished by customizing the relocation constructor of `T`, not by declaring an overload. Also note that because our proposal specifies a move-and-destroy fallback behavior for defaulted move constructors, we need not explicitly specify such fallback behavior for the `std::uninitialized_destructive_move` function template.

§ 7.3 P0023

[P0023R0] uses the syntax `new (dest) >>T(*src)` to construct a `T` object at `dest` having the value held by `*src`. The actual relocation is performed by a function called a *relocator*, introduced in the scope of `T` by a declarator of the form `>>T(T& src)`. Despite looking very different from N4158, P0023 has similar limitations; it cannot be used to safely relocate objects with automatic storage duration, does not prevent use-after-relocation, and does not provide a facility for perfectly forwarding the intent to evaluate `new (dest) >>T(*src)` instead of `new (dest) T(src)`, where `src` is an argument of pointer type.

§ 7.4 P1144R7 and P2786R0

[P1144R7] and [P2786R0] are similar proposals.

- They each introduce the concept of trivial relocatability, in which some types are *implicitly* trivially relocatable, but the author of a class can also explicitly declare that class to be trivially relocatable even if it has nontrivial special member functions.

- They each provide library facilities, known as `std::relocate` and `std::relocate_at`, that take a pair of pointers to T and either perform trivial relocation (when T is trivially relocatable) or destroy the source object after move-constructing the destination object. Such library facilities can be used to speed up operations that are currently expressed exclusively in terms of move plus destroy, e.g., the operation of relocating elements of `std::vector<T>` when the vector's capacity is increased.

The definition of the category of implicitly trivially relocatable types differs slightly between P1144R7 and P2786R0. We do not express an opinion on which definition should be chosen; debate in EWG should resolve this question, and the authors of P1144R7 and P2786R0 are well positioned to argue their respective cases. The same is true for the syntax and semantics of explicitly declaring a class type to be trivially relocatable, which also differ between P1144R7 and P2786R0. Our proposal can build upon the trivial relocatability machinery of either P1144R7 or P2786R0. (Clearly, a class with a *user-provided* relocation constructor — see Part II — will not be trivially relocatable and a diagnostic should be required if the programmer attempts to declare the class to be trivially relocatable.)

Because P1144R7 and P2786R0 both employ pointer-based approaches to performing relocation, they suffer from the same limitations as N4158 and P0023 with respect to automatic variables. Nevertheless, the use of such pointer-based interfaces for relocating objects of dynamic storage duration is compatible with our proposal. For example, if P1144R7 is accepted, our proposal will be to modify the specification of `std::relocate_at` so that when T is not trivially relocatable but has a usable relocation constructor, that constructor will be called in preference to performing a move-and-destroy operation.

§ 7.5 P1029R3

[P1029R3] proposed a pure core language extension to define a move constructor as performing a bitwise copy from source to destination, followed by resetting the source object by copying into it the bit pattern that would be produced by its default constructor (which is required to be `constexpr`).

P1029R3 was made deliberately minimal to have the best possible chance of being adopted into C++23, and its author is no longer pursuing it. In particular, P1029R3 offers no form of nontrivial relocation.

Our proposal offers the semantics of a P1029R3-style relocation:

```
template <class T, int = (T(), 0)>
void P1029R3_relocate(T* from, T* to)
requires std::is_trivially_relocatable_v<T> {
    static constexpr T zero;
    std::memcpy(to, from, sizeof(T));
    std::memcpy(from, &zero, sizeof(T));
}
```

§ 8 Appendix A: Potentially breaking changes

- $x.\sim\tau()$, where x is an automatic variable of type τ , can also disengage $__x\sim$. This rule would make naming x after this line ill formed, unless and until a placement new expression is used to recreate x . This rule would also break some existing code but potentially increase safety by preventing access to a variable whose lifetime has ended. Specifying this rule would be painful: We would need to define the class of expressions we consider to be placement new expressions for the purposes of this rule, and we would also need to introduce another set of control flow rules. (The control flow rules for Part I assume that owning references can be disengaged along some paths of control flow; here, we would need rules that account for owning references being able to become re-engaged.)
- Require callee-destroy for all by-value function parameters, which would allow `reloc` to be applied to function parameters.

§ 9 Appendix B: Alternative syntaxes for forwarding references

We considered three possible approaches to perfectly forwarding owning references. We propose the first approach below but are open to polling to determine the best choice.

§ 9.1 $\tau\sim$ syntax

The approach we propose in this paper is that a function parameter whose declared type is $\tau\sim$, where τ is the name of a template parameter of the function, is a forwarding reference. The main advantage of this syntax is its consistency with the reference collapsing rules in the same way as the current forwarding reference syntax, $\tau\&\&$. Like $\tau\&\&$, the syntax $\tau\sim$ requires only the addition of a special template argument deduction rule to ensure that τ is deduced as a type that will give the appropriate reference type after the collapsing rules are applied to $\tau\sim$.

The $\tau\sim$ syntax has two disadvantages. The first is that all function templates that currently perform perfect forwarding using $\tau\&\&$, including Standard Library function templates, would need to be updated to accept $\tau\sim$; otherwise, they would forward rvalues as xvalues, not as rvalues. The second is that when a programmer wants to write a function template that accepts rvalues, *not* glvalues, of any type and deduces that type, a constraint must be introduced into the declaration, as we have done in our proposed declaration of `std::disengage`. This annoyance very rarely arises in the context of $\tau\&\&$ forwarding references, because few situations arise where a function template must accept *only* rvalues but doesn't care about the types of those rvalues. We anticipate that this annoyance will occur much more frequently if the $\tau\sim$ syntax for forwarding references is adopted.

§ 9.2 T&& syntax

To enable all function templates that currently accept forwarding references to perfectly forward rvalues without any changes to their signatures, EWG could adopt an approach in which $\tau\&\&$ gains the ability to perfectly forward rvalues. However, all such approaches known to the authors have considerable disadvantages. Furthermore, adopting such an approach will not enable such function templates to *automatically* begin supporting rvalue forwarding; while the signatures of such functions would not need to change, the function implementations could not effect rvalue forwarding using the syntax `std::forward<T>(t)`, since such a function call would never be able to disengage `t` and transfer ownership to the result of the function call.

§ 9.2.1 Subapproach 1: Changing the reference collapsing rules

One possible approach that would allow the $\tau\&\&$ syntax to perfectly forward rvalues is to specify that $\tau\&\&$ collapses to $\tau\sim$, not $\tau\&\&$. Unfortunately, this violates the principle of lesser privilege discussed in the Summary of Part I. We do not fully understand the practical implications of such a counterintuitive reference collapsing rule. Standardizing this rule might not be catastrophic for the safety of the language, because a variable whose type is spelled $\tau\&\&$ and turns out to be an owning reference is unlikely to be destroyed unintentionally; the `reloc` operator must be used to transfer ownership to another owning reference. However, having $\tau\&\&$ collapse to $\tau\sim$ would interfere with the declarations of the `std::get` function templates for `std::tuple`. When `std::get<T>` is called on an rvalue of type `std::tuple`, one of whose element types is $\tau\sim$, the declared return type is $U\&\&$, where U is $\tau\sim$. If $\tau\&\&$ is $\tau\sim$, then the result of the call is an rvalue, which it should not be, since the only way to return an rvalue would be to leave the tuple in a partially relocated state. This issue with `std::get` was not immediately obvious to the authors, and other unanticipated issues are likely if this reference collapsing rule is adopted.

§ 9.2.2 Subapproach 2: Changing the reference collapsing rules but only for forwarding references

A variant of subapproach 1 is to retain the natural reference collapsing rules in which $\tau\&\&$ collapses to $\tau\&\&$ but add a special exemption solely for forwarding references: When $U\&\&$ is a forwarding reference and U is $\tau\sim$, the result is $\tau\sim$, while in all other contexts, the result would be $\tau\&\&$. That the meaning of code would depend too much on whether a reference is a forwarding reference is the main disadvantage of this approach.

§ 9.2.3 Subapproach 3: Making reference collapsing ill formed outside forwarding references

A variant of subapproach 2 is to make $\tau\&\&$ collapse to $\tau\sim$ in forwarding reference context and be ill formed in every other context. Such an approach avoids the main disadvantage of the subapproach 1 but suffers from the same issue as subapproach 1 concerning `std::get` and forces writers of generic code to guard against the creation of the $\tau\&\&$ type.

§ 9.2.4 Subapproach 4: Abominable types

Another subapproach for avoiding counterintuitive reference collapsing outside of forwarding references is to specify that $\tau\&\&$ is neither $\tau\sim$ nor $\tau\&\&$ but is adjusted to $\tau\sim$ in a function declaration that uses a forwarding reference. In all other contexts, $\tau\&\&$ would be an *abominable type*, and attempting to declare a variable or evaluate an expression whose type would be $\tau\&\&$ would be ill formed. This subapproach suffers from the same issue with `std::get` as subapproaches 1 and 3 but might avoid the disadvantages of subapproach 3 in other contexts. Unfortunately, if WG21 adopts this subapproach and it later turns out to be untenable, removing the abominable types from the language and specify a different behavior will be difficult.

§ 9.3 A completely different syntax

We could invent a new syntax for forwarding references that would perfectly forward rvalues, such as $\tau\&\&\&$ or $\tau\sim\sim$. This approach would avoid all the disadvantages associated with the $\tau\&\&$ syntax and the second disadvantage associated with the $\tau\sim$ syntax but suffers from a severe disadvantage of its own: It removes design space for more general improvements to perfect forwarding, such as a syntax that would enable forwarding of overload sets or *braced-init-lists*.

§ 9.4 Conclusion on forwarding reference syntax

We propose the $\tau\sim$ syntax because, although it is imperfect, its problems are less severe than those introduced by all known alternatives. The problems with the $\tau\sim$ syntax parallel the problems with the existing $\tau\&\&$ syntax in current C++, which have proven to be tractable.

§ 10 Appendix C: Alternative names

A possible alternative name for *rlvalue* is *dvalue*, the *d* of which connotes permission to *destroy* the referent. The name *dvalue* is analogous to *xvalue*, whereas using the term *rlvalue* is more comparable to referring to *xvalues* as *mvalues*, i.e., connoting the likely (but not certain) fate of the object rather than the permission granted to the holder of the reference.

§ 11 References

- [D2785] Sébastien Bini, Ed Catmur. 2023-01-30. Relocating prvalues.
<https://github.com/SebastienBini/cpp-relocation-proposal/blob/main/relocation.bs>
- [N4158] Pablo Halpern. 2014-10-12. Destructive Move (Rev 1).
<https://wg21.link/n4158>
- [P0023R0] Denis Bider. 2016-04-08. Relocator: Efficiently moving objects.
<https://wg21.link/p0023r0>
- [P0644R1] Barry Revzin. 2017-10-08. Forward without forward.
<https://wg21.link/p0644r1>

[P1029R1] Niall Douglas. 2018-08-07. `[[move_relocates]]`.

<https://wg21.link/p1029r1>

[P1029R3] Niall Douglas. 2020-01-12. `move = bitcopies`.

<https://wg21.link/p1029r3>

[P1144R6] Arthur O'Dwyer. 2022-06-10. Object relocation in terms of `move` plus `destroy`.

<https://wg21.link/p1144r6>

[P1144R7] Arthur O'Dwyer. 2023-03-10. `std::is_trivially_relocatable`.

<https://wg21.link/p1144r7>

[P2786R0] Mungo Gill, Alisdair Meredith. 2023-02-11. Trivial relocatability options.

<https://wg21.link/p2786r0>